

期末報告

simplifiedb

A relational database engine built from scratch in C
SQL subset · B-Tree indexing · Data persistence

組員: 陳達偉 , 孫義廷

Table of contents

1. Modules
2. Data Structure
 - Page
 - B+ Tree
3. How it works
4. Demo

Modules

REPL

Read-Eval-Print Loop

Executor

INSERT / SELECT / UPDATE / DELETE

B-Tree

index, search, split / merge

Cursor

B-Tree position pointer

Pager

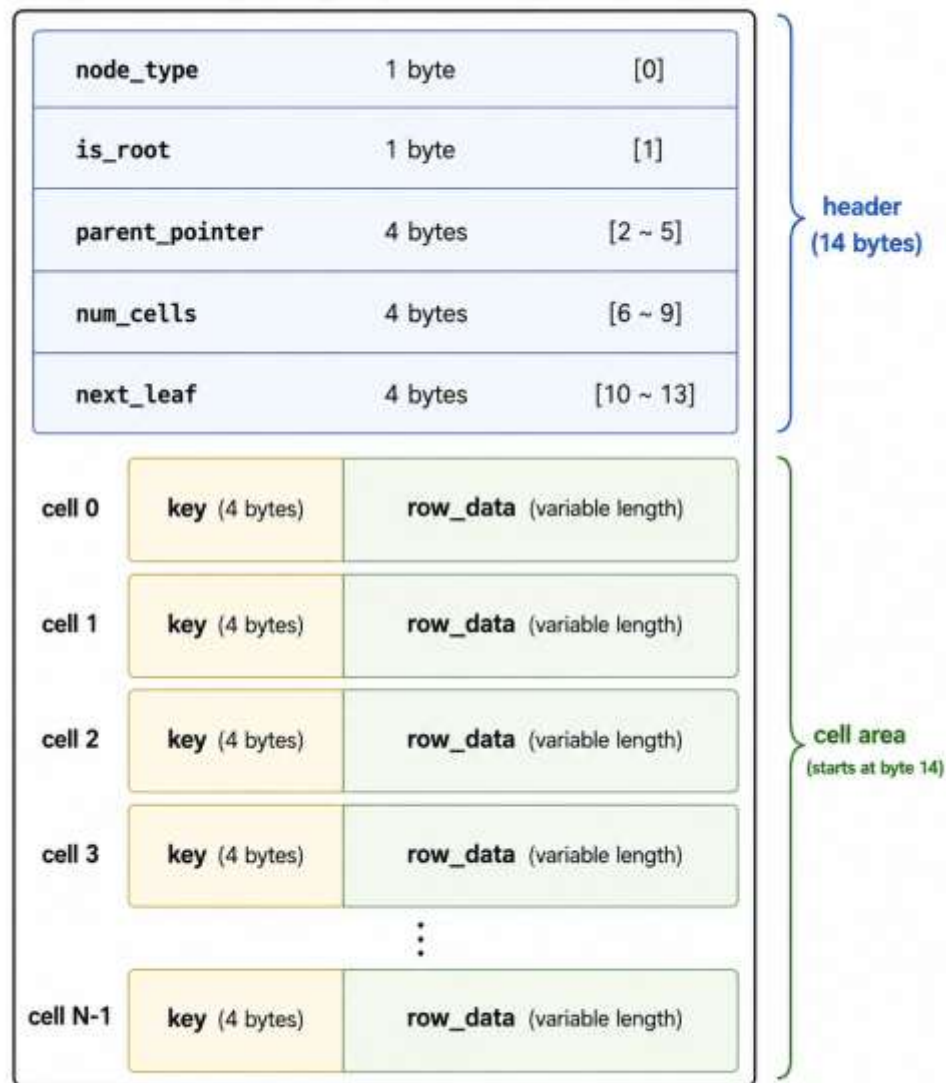
Memory Page Manager

Catalog

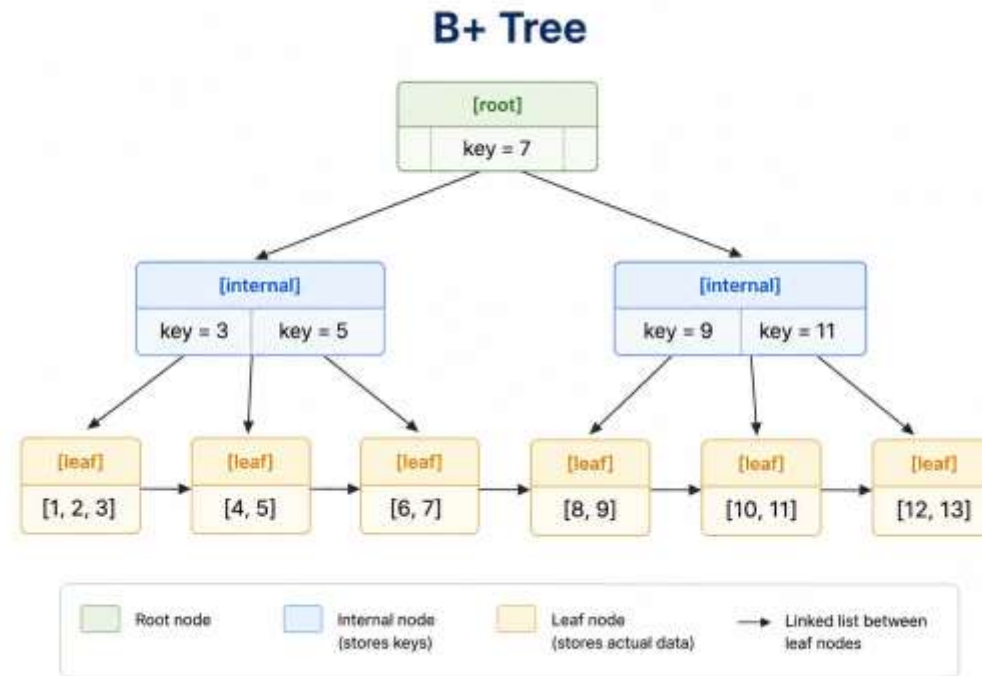
table schema

Page

Page (4096 bytes)



B+ Tree



- internal node — routing key only, decides direction
- leaf node — stores row data, linked list between leaves
- search: $O(\log n)$ · full scan: walk leaf linked list

How it works — Insert (1/2)

1. Tokenize input

```
char *tok = strtok(buf, "\t\r\n");
while (tok && n < MAX_TOKENS) {
    tokens[n++] = tok;
    tok = strtok(NULL, "\t\r\n");
}
```

2. table_open → catalog lookup → malloc Table

```
Table *table_open(Database *db, const char *name) {
    const int idx = catalog_find(&db->catalog, name);
    Table *t = malloc(sizeof(Table));
    t->meta = &db->catalog.tables[idx];
}
```

3. Compute page layout

```
t->cell_size = LEAF_NODE_KEY_SIZE + t->meta->row_size;
t->max_cells = (PAGE_SIZE - LEAF_NODE_HEADER_SIZE) / t->cell_size;
t->right_split_count = (t->max_cells + 1) / 2;
t->left_split_count = (t->max_cells + 1) - t->right_split_count;
```

How it works — Insert (2/2)

4. table_find → recursive + binary search → Cursor

```
// recursive
if (get_node_type(child) == NODE_LEAF)
    return leaf_node_find(table, child_page, key);
return internal_node_find(table, child_page, key);

// binary search
while (lo < hi) {
    const uint32_t mid = (lo + hi) / 2;
    const uint32_t k = *leaf_node_key(table, node, mid);
    if (key < k) hi = mid;
    else      lo = mid + 1;
}
```

5. Shift cells, write at cursor position, update num_cells

```
for (uint32_t i = num_cells; i > cursor->cell_num; --i)
    memcpy(leaf_node_cell(table, node, i),
           leaf_node_cell(table, node, i - 1),
           cursor->table->cell_size);
*leaf_node_num_cells(node) += 1;
memcpy(leaf_node_value(table, node, cursor->cell_num),
       row_data, cursor->table->meta->row_size);
```

6. Mark dirty, flush on .exit

```
for (int i = 0; i < MAX_PAGES; ++i) {
    if (pager->pages[i] && pager->is_dirty[i])
        pager_flush(pager, i);
}
```

報告完畢，感謝聆聽！